Chapter 14.3: From Design to Implementation

About this page:

Questions answered: How to design larger programs? How to proceed from a design to a functioning program?

Topics: Let's look at two opposite design methods, the Top-Down and Bottom-up approaches, and how (and when) we proceed from designing to implementation. We consider how the program can be implemented in parts so that the whole can be tested even before all the parts exist.

What will I do? Primarily reading - the UML plan from the previous section can be used as reference, as needed.

Estimated difficulty level: Easy, no assignments, but on the other hand, a lot of new terms. Understanding them would require making your own design..

Rough estimate of workload: Roughly an hour.

Points available: 0

The structure of larger programs and APIs

A small, simple program may consist of a few classes that use each other's services. As more classes are added, the complexity of the program grows and it becomes harder to keep track of how the program as a whole works. When the classes also depend on one another, changes in one place may affect another. In order to make programs just a little bigger possible, the amount of information that is being processed at a time needs to be managed in some way.

The term *abstraction* has come up many times earlier in the course material. Abstraction generally means that some aspect of a problem or solution can be looked at and used by focusing solely on its essential features and ignoring deliberately its internal implementation. We

call this action *forming abstractions*. The related term for this is *information hiding* where we **intentionally** hide implementation details..

Modularization refers to a process by which an entity (program, part of a program, algorithm, etc.) is divided into clear subparts that can be implemented independently of each other. These subparts are called *modules*. Each module also has a clear *interface* through which other modules can work with it and it works with other modules. The module also has a detailed program code *implementation*.

The term *encapsulation* refers to the aggregation of data, and the functions or methods for processing it, into a coherent whole.

Although the above terms resemble each other, they do not exactly mean the same. For example, information hiding and encapsulation seem similar, but the former is seen as a design principle, whereas encapsulation is a programming language mechanism.

Programming languages support encapsulation, modularisation, and the forming of abstractions in different ways. Scala offers several tools for grouping information and modularizing.

- Classes are often grouped through *packages*. Packages include classes or packages that together form an entity for which we want to give a name. Packages form a *hierarchy*.
- Classes, traits, and methods can all include class definitions. Thus these structures can, in addition to packages, be used for grouping of classes.

From design to implementation

Where should we start when implementing a big program? One of the major problems when designing and implementing programs is that a program has a large number of functionalities distributed across classes with a number of dependencies. A bigger program cannot be implemented by developing it as a single whole, relying on the assumption that all the parts eventually fit together.

So let's look at two opposite design methods, Top-Down and Bottom-up approaches, and how we proceed from designs to implementation.

Top-down design

In top-down design, the work starts from the whole system level and reflects on what the system does at the highest abstraction level. The aim is to find and design subsystems of the next level without deciding anything about their implementation yet. The subsystem is at this stage a kind of ”black box” that provides all the necessary services, but its implementation is unknown to us (i.e., it has not been designed nor implemented yet).

For example, we can split Timeliner's (Section 14.1) operational logic into three parts related to user interface, main logic, and announcements part. Separating user interface from the other parts is generally a good principle.

The above procedure is repeated at the level of each black box subsystem. The subsystem is described by using smaller parts. These may again be some subsystems or collections of yet undefined classes. However, their responsibilities and dependencies from other classes can once again be set even though we do not have designed/implemented these actual classes yet. Similarly, each identified subsystem can be partitioned in the same way,

The question arises as to when does this partitioning end? We might think we are proceeding to the level of individual code lines, but this is generally not meaningful. Usually, at some point, we realize that there is a ready-made implementation for the subsystem required, either by using something in a library or by re-using the code we are making ourselves, or by recognizing classes and methods we need to implement ourselves.

In practice, during the design we define the external interfaces of the systems before they are implemented. When determining the interfaces, we consider how sub-systems at the same abstraction level can use each others' services. Once the interfaces are properly configured, we can implement and test different subsystems / classes irrespective of each other. Subsystem A does not need to know the implementation of subsystem B; it is enough to know B's interface.

Repeating - No pain no gain

The first attempt may be disappointing. At the same time, however, we have also learned something new. Thus, if the design process is repeated, the result will almost certainly improve. Probably the plan needs to be refined several times before it is feasible. When implementing the design, one frequently learns new aspects of the necessary tasks and component responsibilities, which should have been noticed earlier. The process can well be repeated from top to bottom while performing corrections. Remember also that if the design fails, it is better to abandon it than proceeding stubbornly.

Top-down implementation

Despite creating a good design, it is almost never a good idea to fully implement a program before starting to test it. A much better option is to build the program piece by piece and test its functionality one by one. In practice, this requires that it is possible to compile and execute the program, even though some parts of it have not yet been implemented. The basic idea behind such Top-Down implementation is that the program code has initially only the structure: classes and methods, which may be empty or simplified in their implementation. Below we present different ways you can do this: Dummy, Stub, and Mock.

As a result of the top-down design, we have class interfaces and an idea of the packages they belong to, but there is hardly any implementation for the classes (excluding re-used code, e.g. library classes). In order to begin coding a single class, all the other classes that it needs are replaced with classes that have the desired interface, but no functionality. This enables

compiling the class without unnecessary error messages. As we will see later in Section 16.2_, the class functionality can not only be compiled but also tested even if the implementation of required classes is missing..

We can use different types of ”substitute classes” instead of the correct implementation. A substitute class is needed to enable code compilation, but it is important that the functionality of the code can also be tested at an early stage. Testing and testing tools will be explained in more detail in Section 16.2_.

Dummy

Dummy classes are the simplest of the substitute classes. In practice, it is a class with the simplest implementation for its interface. Scala makes coding such classes quite easy, because the bodies of the required methods can be replaced by using the tag ”???”. This tag is, in fact, a method call with the return value of type `Nothing`, which, however, invokes an exception by throwing an exception, if it is called. (We discuss exceptions more in Section 15A) You have seen multiple such empty methods in the exercises in the Programming 1 course.

Nothing-type

Nothing (http://www.scala-lang.org/api/current/scala/Nothing.html) is at the other end of the Scala type hierarchy than class Any, which was introduced in Section 7.3 (https://plus.cs.hut.fi/o1/2020/w07/ch03/). `Nothing` is the subtype of all types (even `Null`). Nothing-objects cannot be instantiated. However, due to the special definition of function `???` that returns Nothing, this function can replace any return value.

Stub

`Stub` is already a substitute class for testing. It has minimal internal functionality, which allows it to be used when developing new code as part of the ongoing continuous small testing activity. Similarly as Dummy, it implements the desired interface; however, it returns ”prespecified” responses to the calling code. Stubs can also be built so that they record the method calls, constructor parameters, etc. This information can often be used as help in testing to find out if the class using the stub is working correctly.

In the example below, a stub simulates a missing database class by acting as a class that always ”finds” what the caller wants. The stub database works from the perspective of the class that uses it, but it does make a lot of simplification. Consider, in which situations would the part that needs this code work incorrectly?

```
// The interface class that the actual class would implement
trait CustomerDB {
  def getCustomerByName (name: String): Option [Customer]
}

// A stub that "replaces" the actual class
class CustomerDBStub extends CustomerDB {
  def getCustomerByName (name: String): Option [Customer] = {
    // does not really access the database, but creates this "on the fly"
    Some (new Customer (name))
  }
}
```

Mock

Mock is a more developed substitute class. Mock is essentially different from stubs in the sense that we define in advance how it will be used in testing. In other words, when the stub always returns the same value or structure, Mock can manage with different situations. It returns the desired ”canned” response for each individual test value that is defined for it.

In the example below, the Mock object can be told that in testing phase its method `getCustomerByName` will be called either with ”Mikko” or ”Petteri” as the String parameter, and then it should return the corresponding created `Customer` object. The `expect` method just looks at which input value it receives and `returns` the object.

If during the test the method is called with some other parameter, or some other method of the mock is called first, the mock gives a warning and rejects the running test (implementation not visible here)

```
trait CustomerDB {
  def getCustomerByName(name:String): Customer
}

class MockedCustomerDB extends CustomerDB {

  val mockedGetCustomer = mockFunction[String, Customer]
  val Mikko   = new Customer("Mikko")
  val Petteri = new Customer("Petteri")

  mockedGetCustomer.expects("Mikko").returns(Mikko)
  mockedGetCustomer.expects("Petteri").returns(Petteri)
  mockedGetCustomer.expects("Mikko").returns(Mikko)

  def getCustomerByName = mockedGetCustomer
}
```

Bottom-up design

The Bottom-up design strategy proceeds in the opposite direction than Top-Down method. It begins from the individual parts and combines them layer by layer to larger entities. Often this means designing and implementing ”tool classes” which will be used to support implementing

many other classes.

In principle, programming and testing of individual classes could be started immediately. However, this should not be carried out too much separated from other designing. Combining parts with other code many later be difficult and the result may be confusing, difficult to modify and maintain.

An important advantage of Bottom-up design is that it supports code re-use. The old parts are considered as bases for design, and they can be used with small modifications or as such. In this aspect, Bottom-up design differs from Top-Down design, where the resulting interfaces and the requirements for a single component may not necessarily fit together with the old code as such.

Reality

In reality, it is worth combining the different approaches to achieve the best result.

A practical solution in the design process is to combine and alternate Top-Down and Bottom-up design methods so that a clear and maintainable structure for the software is created. At the same time, the lowest level components can be reused and their interfaces are clear and usable. In practice, it is necessary to iterate the design for some modules several times until a good solution is found. This could be called *Meet-in-the-middle design*, for instance.

Top-down design avoids modular incompatibility problems and unnecessary or complex dependencies and gives an overall picture of the project. The Bottom-up approach enables, among other things, code reuse and allows for the design of individual classes without a dictated structure.

> **Tip: TODO**
>
> When the project has empty classes, substitute code, and generally something unfinished, it's a good idea to mark this code with a comment to avoid future confusion. Commonly used words are TODO and FIXME, which are automatically recognized by some :term:`IDE`s, showing them, for example, in the unfinished work list.

The quality of the design

The quality of the created package and class structure relate to the terms *cohesion* and *coupling*.

High cohesion

The different parts of the program are responsible for the different tasks in the program. If there are a number of different tasks in the class that depend only loosely on each other, the probability that this code could be reused is quite small. The more clearly a single class handles

an individual, clearly defined task, the easier it is to reuse. The same rule also applies to methods and larger modules. The smaller the units of the code are and the more clearly they are defined, the greater the cohesion.

You may want to consider the Timeliner project's job descriptive class, for example. Is there a number of tasks in the class that one could outsource to a class that would do just that job well?

Loose Coupling

Take a look at the UML diagram you built in the last section. Some of the classes in the diagram have a number of dependencies on other classes, and it is clear that they also use a large number of each other's methods in communicating information with each other. If you need to make a change in one class, another class and classes depending on it may easily need to be changed, too.

Coupling means how wide and complex the interface between two classes is. If there are a few clear methods in the interface, and classes need to know as little as possible about each other's implementation, there is a loose connection between the classes.

It is worth noting that achieving high cohesion and loose coupling may require more work in the beginning and produce larger code. However, the benefits will follow later in the code life cycle.

Other remarks

Unnecessarily complex dependencies between the classes are not desirable. In the UML diagram, other potential problems can also be identified. Situations where very many routes can be used to access the same information, and especially if there are loops in the diagram, may be signs of too much complexity. Consider whether all references are needed or whether the information can be accessed by other means, as well.

In general, it is good that a dependency is one-way. For example, the relationship between the user interface and the program logic should always be constructed so that the user interface depends on the logic, but not vice versa. In this case, the user interface can be modified and large features can be added without the need to make changes to the logic code.

It should also be emphasized that there is no best solution to design problems. It is also worth noting that design goals can (and often are) contradictory, so the designer sometimes has to make compromises.

Summary

- In the design, the program should be divided into some subsystems and, as far as possible, avoid unnecessary dependencies between them.
- Top-down design works by dividing the program's responsibilities into modules, continuing down to the individual classes.
- The Bottom-up method designs the classes which are appropriate to the topic and combines them into subsystems.

- Both approaches have their weaknesses. There is no one method that will always achieve a good design result, but in practice the methods are combined and a part produced by one system can be redesigned by employing another. Iterating the design is a good idea — one often learns more about the problem area.
- There is no universally valid way to find out whether a solution is ”good” or ”bad”. There are different solutions to the same problem and it is unlikely for two programmers to produce a similar solution to a non-trivial problem.
- When writing and testing a program, classes can be replaced by substitute classes. Different types of substitute classes meet different needs.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately

⚠ This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

This course has been archived (Thursday, 1 June 2023, 12:00).

Show anyway

Time spent: (*) Pakollinen Please estimate the total number of minutes you spent on this chapter (reading, assignments, etc.). You don’t have to be exact, but if you can produce an estimate to within 15 minutes or half an hour, that would be great.

10

”I feel that I have understood the most important things in this chapter.” (*) Pakollinen

- (●) fully agree
- ( ) somewhat agree
- ( ) somewhat disagree
- ( ) fully disagree
- ( ) I’m unable to answer or don’t want to comment.

Submit an update